

Robust Diacritics Restoration and Detection of Incorrect Diacritics in Romanian

Lungu Andra Iasmina, Budai Marius Gabriel, Luca Andrei, Dinca Georgian, Paicu Eliza

May 13, 2026

Contents

1	Introduction	2
2	Project Overview	2
2.1	Motivation	2
2.2	Goals	2
3	System Architecture	3
4	Data Generation Pipeline	3
4.1	Motivation and Design	3
4.2	Degradation Scenarios	4
4.3	Implementation Details	4
4.4	Data Sources	4
5	Diacritics Restoration Model	4
5.1	Model Choice: ByT5	4
5.2	Training Procedure	5
5.3	Inference	5
5.4	Post-processing	5
6	Per-character Confidence Module	6
6.1	Overview	6
6.2	Extraction from Model Softmax	6
6.3	Mapping to the Post-processed Text	6
6.4	Design Rationale	6
7	REST API	7
7.1	Architecture	7
7.2	Endpoint Reference	7
8	Evaluation	7
8.1	Methodology	7
8.2	In-Distribution Results	8
8.3	Out-of-Distribution Results	8
8.4	Per-Confusion Restoration Accuracy	9
8.5	Detection Module Evaluation	9
8.6	Error Analysis	9
8.7	Training Dynamics	11
9	Technologies and Dependencies	12
10	Deliverables	12

1 Introduction

Romanian digital text is frequently affected by the absence or misrepresentation of diacritical marks — a consequence of legacy encodings, OCR processing, informal typing, and keyboard limitations.

This project presents an end-to-end system for robust diacritics restoration and detection of incorrect diacritics in Romanian. The core of the system is a fine-tuned ByT5 sequence-to-sequence model that operates at byte level, taking degraded Romanian text as input and producing a fully diacritized version as output. Byte-level processing makes the model inherently robust to orthographic noise and independent of a fixed vocabulary, which is a key advantage for a language with rich diacritical variation.

Alongside the restoration model, the system includes a data augmentation pipeline that synthetically generates training data by applying six types of controlled degradation to clean Romanian text, covering full diacritic stripping, partial stripping, wrong-diacritic substitution, legacy cedilla encoding, and OCR-style noise. A confidence module reads the per-step softmax distributions produced by the model during generation and aggregates them into per-character probabilities, which the demo surfaces visually so the user can see which positions the model was less sure about. The complete system is exposed through a REST API built with FastAPI, including an interactive demo interface.

Keywords: diacritics restoration; Romanian NLP; ByT5; sequence-to-sequence; data augmentation; OCR noise; per-character confidence; error detection; text normalization

2 Project Overview

2.1 Motivation

Romanian is a language with five diacritical characters: ș (s with comma below), ț (t with comma below), ă (a with breve), â (a with circumflex), and î (i with circumflex). The absence or misrepresentation of these characters significantly degrades text quality and can alter meaning. Common sources of diacritic errors include:

- Copy-paste from systems that do not support UTF-8 diacritics, replacing accented characters with their base forms (e.g. ș→s, ț→t).
- OCR processing of scanned Romanian documents, introducing character-level substitutions and split/merge errors.
- Legacy encodings that use cedilla variants (ş, ţ) instead of the correct comma-below forms (ș, ț), a widespread issue in older Romanian digital text.
- Informal typing habits on mobile devices or non-Romanian keyboard layouts.

2.2 Goals

The system aims to:

- Restore missing or incorrect diacritics in Romanian text with high accuracy across multiple degradation types.
- Estimate per-character uncertainty so that downstream consumers can flag positions for human review.
- Provide a reproducible data generation pipeline that produces realistic training and evaluation data.
- Expose all functionality through a documented REST API suitable for integration into other systems.

3 System Architecture

The system is organised as a Python package with a FastAPI web service at its core. The architecture follows a modular pipeline in which each component can be used independently or in sequence. Figure 1 summarises the data flow at inference time.

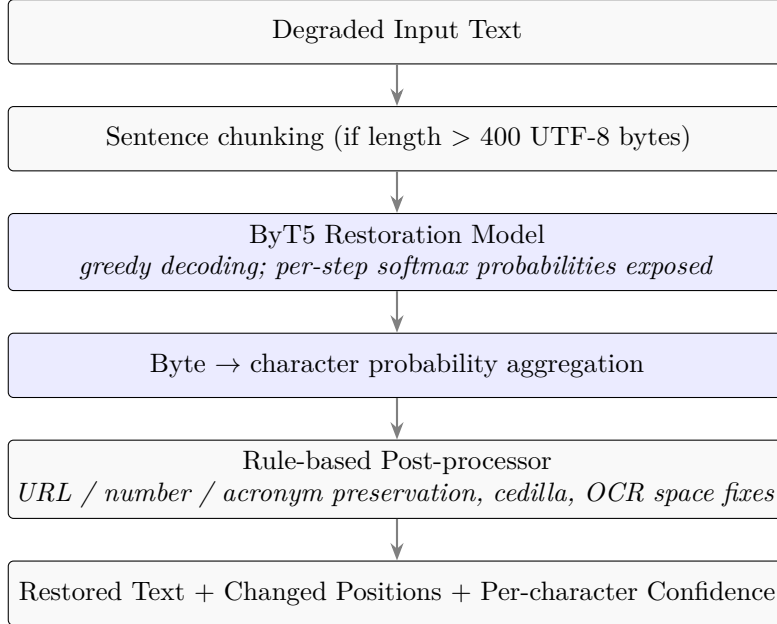


Figure 1: Inference-time data flow. Blue nodes are model-derived components; gray nodes are deterministic pre- and post-processing stages.

Inputs longer than 400 UTF-8 bytes are split at sentence boundaries and restored chunk-by-chunk; per-character confidences are concatenated in the natural reading order. The model performs greedy decoding so that softmax probabilities are well-defined at each generation step; these probabilities are aggregated across the bytes of each UTF-8 character by taking the product, yielding the joint probability of producing the whole character correctly. The post-processor runs as a deterministic rule layer on top of the model output, and the final per-character confidences are aligned to the post-processed text.

4 Data Generation Pipeline

4.1 Motivation and Design

Because large-scale manually annotated corpora of Romanian text with diacritic errors are not readily available, the project generates training data synthetically by applying controlled degradation to clean Romanian text. The generator in `app/degradation.py` implements six distinct degradation scenarios, each modelling a different real-world error source.

4.2 Degradation Scenarios

Table 1: Degradation scenarios.

Scenario ID	Description	Typical Source
strip_all	All diacritics removed: $\text{ș} \rightarrow \text{s}$, $\text{ț} \rightarrow \text{t}$, $\text{ă} \rightarrow \text{a}$, $\text{â} \rightarrow \text{a}$, $\text{î} \rightarrow \text{i}$	Keyboard without RO layout
strip_partial	Each diacritic removed independently with probability p	Inconsistent typing
wrong_diacritics	Wrong diacritics introduced (e.g. $\text{â} \leftrightarrow \text{î}$, $\text{ă} \leftrightarrow \text{â}$)	Autocorrect errors
cedilla	Correct $\text{ș}/\text{ț}$ replaced with legacy cedilla $\text{ş}/\text{ţ}$	Legacy ISO-8859-2 encoding
ocr	Character substitutions plus random space splits/merges	OCR from scanned documents
mixed	Random combination of the above per sample	Recommended for training

4.3 Implementation Details

The core function `degrade_text(text, config)` applies one scenario to a single string and returns the degraded text along with the scenario identifier. Batch generation is handled by `generate_samples()`, which iterates over all six scenarios for every input sentence, producing up to six degraded variants per sentence.

Each returned `DegradedSample` contains the degraded input, the clean target, the scenario name, and a list of character positions that were changed (useful for evaluation). Character positions are computed by a simple character-level diff; for OCR scenarios that change text length, the changed positions list is left empty because alignment is ambiguous.

4.4 Data Sources

The primary data source for training is RoLargeSum [3], a large Romanian news summarisation dataset hosted on HuggingFace (`avramandrei/RoLargeSum`). Clean article and summary texts are split into sentences using a punctuation-based splitter and filtered to the range 20–512 characters. A local JSONL cache (`data/train.jsonl`) is used for offline training. The training set used for the final model was a 40,000-example stratified subsample of a 288,000-example generated dataset; the validation set contains the remaining examples.

5 Diacritics Restoration Model

5.1 Model Choice: ByT5

The restoration component is built on ByT5 (Byte-level T5) [1], a sequence-to-sequence transformer that operates directly on UTF-8 bytes rather than subword tokens. This architecture is particularly well-suited for diacritics restoration:

- It does not depend on a fixed vocabulary and therefore handles any Unicode character natively, including all Romanian diacritical forms.
- It is robust to orthographic noise: character-level errors that would fragment a subword tokenizer into out-of-vocabulary pieces are handled naturally.
- The restoration task is a near-identity mapping (most characters are unchanged), which ByT5 handles efficiently through its copy mechanism.

5.2 Training Procedure

Training is implemented in `scripts/train.py` using the Hugging Face Transformers library [4]. The script takes a JSONL training file and a JSONL validation file, each containing (input, target, scenario) triples generated by the pipeline of Section 4. The final training run used the configuration in Table 2.

Table 2: Training configuration.

Hyperparameter	Value	Notes
Base model	google/byt5-base	~580M parameters
Training examples	40,000	Stratified mix of all six scenarios
Epochs	1	Further epochs gave no measurable CER improvement
Optimizer	AdamW	Weight decay 0.01
Learning rate	3×10^{-4}	Linear warmup (500 steps), linear decay
Effective batch size	16	4 per-device $\times$ 4 gradient-accumulation
Max input / target length	512 bytes	~200–400 Romanian characters
Mixed precision	bf16	RTX 4090 Mobile, 16 GB VRAM
Best-checkpoint criterion	Minimum validation CER	<code>load_best_model_at_end = True</code>

During training, evaluation runs every 500 steps on a 200-example subset of the validation set (for speed). At the end of training, a full per-scenario evaluation runs on a 3,000-example stratified sample of the validation set and is written to `eval_by_scenario.csv`. All training metrics (loss, CER, WER) are logged to `training_log.csv` using a plain `csv`-module callback; no external experiment tracking tools are required.

5.3 Inference

The `DiacriticsRestorer` class in `app/predict.py` wraps the trained checkpoint and exposes a `restore(text)` method. Inputs longer than approximately 400 UTF-8 bytes are automatically split at sentence boundaries, each chunk restored independently, and the results concatenated. The method returns a `RestorationResult` containing the restored text, a list of character positions in the restored text that differ from the input, an overall confidence score (mean per-character probability), and a parallel array of per-character probabilities.

Decoding is performed greedily (`num_beams = 1`) rather than with beam search. Greedy decoding makes per-step softmax probabilities directly available and trivially mapped to the chosen token; beam search returns only aggregate sequence scores that cannot be mapped to individual characters in a principled way. For the restoration task — which is close to identity for most positions — greedy decoding empirically loses no measurable quality compared to beam search while running roughly four times faster.

5.4 Post-processing

A rule-based post-processor (`app/postprocessor.py`) runs after the model to fix systematic issues that the model cannot learn reliably:

- **OCR artefacts.** Token-level character substitutions ($l \leftrightarrow 1$, $o \leftrightarrow 0$, $rn \leftrightarrow m$, $ii \leftrightarrow u$) are corrected by comparing the model output to the original input token by token. Multiple consecutive spaces are collapsed.
- **Protected tokens.** URLs, email addresses, numbers, and ALL-CAPS acronyms are copied verbatim from the input back into the output, preventing the model from altering them.
- **Cedilla cleanup.** Any residual cedilla variants ($\text{\textcircled{S}}$, $\text{\textcircled{T}}$) in the output are replaced with the correct comma-below forms ($\text{\textcircled{S}_}$, $\text{\textcircled{T}_}$).

6 Per-character Confidence Module

6.1 Overview

After restoration, the system reports a per-character confidence score for every character of the output. The score quantifies how certain the model was about emitting that specific character. Low-confidence positions are surfaced visually in the demo (Section 7) and can be used by downstream consumers to flag ambiguous corrections for human review. This satisfies the “probability/entropy per character” requirement of the project specification using the model’s own probability estimates rather than an external language model.

6.2 Extraction from Model Softmax

During generation with `return_dict_in_generate=True` and `output_scores=True`, the model returns a `scores` tuple containing one logits tensor per generation step. For each step:

1. Apply softmax to obtain a probability distribution over the model’s 256 + 3 byte vocabulary.
2. Read off the probability of the byte that the model actually emitted at that step.

This yields one probability per generated byte. Because ByT5 emits UTF-8 bytes and Romanian diacritic characters (ă, â, î, ș, ț, and their uppercase forms) are two-byte sequences, multi-byte characters span multiple generation steps. To obtain a single probability per character, the byte-level probabilities of the bytes that form each character are multiplied:

$$P(\text{char}) = \prod_{i=1}^n P(\text{byte}_i \mid \text{prefix})$$

This is mathematically the joint probability of correctly emitting every byte of the character, which is the appropriate aggregation for our purpose. The overall confidence reported alongside the restored text is the arithmetic mean of the per-character probabilities.

6.3 Mapping to the Post-processed Text

When the post-processor modifies the model output (for example, replacing residual cedilla forms or restoring URLs from the input), per-character confidences are re-aligned to the final text. For length-preserving changes — by far the common case — characters that the post-processor did not touch retain their model-derived confidence, and characters that the post-processor rewrote are assigned a confidence of 1.0 because they are produced by deterministic rules. For length-changing changes (uncommon in practice), the system falls back to a uniform confidence value equal to the overall mean.

6.4 Design Rationale

An earlier iteration of this module used an external character-level n-gram language model to estimate per-position uncertainty via Shannon entropy over a confusion set. That approach was replaced by the model-internal softmax approach above for three reasons:

1. **No conceptual gap.** The model already has a probabilistic view of every character it produces. An external LM duplicates that effort and may disagree with the model in ways that obscure rather than clarify uncertainty.
2. **Coverage.** A character n-gram LM trained on reference text would systematically flag rare-but-correct Romanian content — proper nouns, place names, foreign-language quotes — as low-confidence even when the model is sure. Model-derived probabilities do not have this failure mode.

3. **Cost.** Loading and querying an external LM at inference adds latency and memory; the model-derived scores are essentially free.

7 REST API

7.1 Architecture

The service (`app/main.py`) is implemented with FastAPI [5] and follows an async lifespan pattern: the ByT5 model is loaded once at startup and kept in memory for the lifetime of the process. All request and response schemas are defined with Pydantic, providing automatic validation and OpenAPI documentation at `/docs`.

7.2 Endpoint Reference

Table 3: REST API endpoints.

Method	Endpoint	Description
GET	<code>/</code>	Demo frontend (HTML interface)
GET	<code>/health</code>	Reports model load status
GET	<code>/scenarios</code>	Lists available degradation scenario IDs
POST	<code>/restore</code>	Restore diacritics; returns restored text, changed positions, overall confidence, per-character confidences, and post-processor change descriptions
POST	<code>/degrade</code>	Degrade one or more clean texts using the §4 pipeline
POST	<code>/generate-dataset</code>	Generate a full (input, target) dataset as JSON or streamed JSONL
POST	<code>/generate-dataset/stats</code>	Aggregate statistics on a generated dataset without downloading content
POST	<code>/postprocess</code>	Apply the rule-based post-processor to an existing model output

The `/restore` endpoint is the primary inference entry point. Its response schema includes both an overall confidence score and a parallel array of per-character probabilities, allowing the frontend to render character-level confidence highlighting without additional API calls.

8 Evaluation

8.1 Methodology

The model is evaluated on two complementary datasets. The **validation set** (`data/val.jsonl`, generated by the pipeline of Section 4 from RoLargeSum news text, of which a 3,000-example stratified sample is used for headline numbers) measures in-distribution performance. The **gold set** (`data/gold.jsonl`, 360 hand-verified examples derived from Romanian Wikipedia) measures performance on out-of-distribution text from a domain not represented in the training data.

The gold set was constructed by sampling random Romanian Wikipedia article summaries via the Wikimedia REST API, filtering for diacritic-rich sentences, deduplicating against the training targets (no overlap was found), and applying the same six-scenario degradation pipeline used for training. The resulting candidates were manually inspected, with entries removed if the source text contained typographical errors, foreign-language interpolations, or non-natural Romanian phrasing.

Both datasets are evaluated using the same script (`scripts/evaluate.py`) which produces:

- overall and per-scenario CER and WER,

- per-confusion-pair restoration accuracy,
- detection precision, recall, and F1 at character-position level,
- ranked error examples for qualitative analysis.

CER and WER are computed as Levenshtein edit distance over characters and whitespace-separated tokens respectively, divided by the target length. Per-confusion accuracy is computed only on length-preserving examples (the OCR scenario, which can change text length through space-split / space-merge errors, is excluded from this metric and noted separately). Detection metrics treat each character position as a binary classification of whether the position requires correction relative to the input.

8.2 In-Distribution Results

Table 4: Validation set — CER and WER per scenario ($n = 3,000$).

Scenario	n	CER	WER
cedilla	500	0.06%	0.31%
ocr	500	0.15%	0.88%
strip_partial	500	0.22%	1.21%
strip_all	500	0.40%	2.14%
wrong_diacritics	500	0.50%	2.67%
mixed	500	0.61%	3.11%
Overall	3,000	0.32%	1.70%

In-distribution performance is strong across all six degradation scenarios, with no scenario exceeding 0.61% CER. The `cedilla` scenario is effectively solved, which is expected: cedilla restoration is a deterministic substitution that the post-processor handles even without the model. The hardest scenario, `mixed`, contains the broadest variety of degradations per example and consequently the broadest mix of error types — but it still resolves to under 1% CER.

8.3 Out-of-Distribution Results

Table 5: Gold set — CER and WER per scenario ($n = 360$).

Scenario	n	CER	WER
cedilla	60	0.44%	2.16%
ocr	60	0.66%	3.47%
strip_partial	60	0.85%	4.60%
strip_all	60	1.00%	5.44%
wrong_diacritics	60	1.37%	7.33%
mixed	60	1.53%	7.97%
Overall	360	0.98%	5.16%

On the out-of-distribution Wikipedia data, overall CER rises by a factor of roughly three (0.32% $\rightarrow$ 0.98%) and overall WER by a similar factor (1.70% $\rightarrow$ 5.16%). The relative ordering of scenarios is preserved: `cedilla` remains easiest, `mixed` remains hardest. The size of the gap is modest by the standards of OOD evaluation: the model continues to operate well below the 5% CER threshold that would indicate substantial generalisation failure, and the qualitative character of its outputs remains correct Romanian text.

8.4 Per-Confusion Restoration Accuracy

The model’s accuracy at restoring each specific degradation type can be measured by counting how often, when the input contained a particular degraded character, the model produced the correct replacement at that position.

Table 6: Per-confusion accuracy on validation and gold sets. Capital-letter confusions on the gold set have very small sample sizes ($n \leq 4$) and are omitted; they are reported in the full evaluation output but should not be over-interpreted.

Degraded $\rightarrow$ Correct	Val n	Val acc.	Gold n	Gold acc.
s $\rightarrow$ ș	1,175	97.0%	163	90.2%
t $\rightarrow$ ț	1,859	97.1%	106	92.5%
i $\rightarrow$ î	1,269	97.3%	145	75.2%
a $\rightarrow$ ă	3,650	96.1%	232	85.8%
a $\rightarrow$ â	495	94.3%	60	93.3%
ș $\rightarrow$ ș (cedilla)	340	100.0%	40	100.0%
ț $\rightarrow$ ț (cedilla)	567	100.0%	25	100.0%

Cedilla restoration is perfect on both datasets, as expected: the post-processor handles this case deterministically. The most informative drops between in-distribution and out-of-distribution performance are in the lowercase circumflex and breve restorations: i $\rightarrow$ î drops by 22 percentage points (97.3% $\rightarrow$ 75.2%) and a $\rightarrow$ ă drops by 10 points (96.1% $\rightarrow$ 85.8%). These drops are concentrated in specific text characteristics discussed in Section 8.6.

8.5 Detection Module Evaluation

Table 7: Detection F1 by scenario on validation and gold sets.

Scenario	Val F1	Gold F1	Δ
wrong_diacritics	98.98%	96.04%	−2.94
mixed	99.25%	96.05%	−3.20
strip_all	96.23%	88.52%	−7.71
strip_partial	96.30%	83.96%	−12.34
cedilla	98.66%	87.01%	−11.65
ocr	92.43%	47.37%	−45.06
Overall	98.25%	92.87%	−5.38

Detection performance is strong on both datasets overall, with the model correctly identifying which positions need correction more than nine times out of ten on the out-of-distribution gold set. The dominant exception is the ocr scenario on the gold set, where F1 collapses from 92.4% to 47.4%. Decomposing this number reveals that recall remains at 100% (the model misses no genuine errors) but precision drops from 86% to 31% — the model flags many positions as needing correction that did not in fact need it. In other words, on noisy out-of-distribution text the model becomes over-eager and “corrects” characters that were correct in the input. This connects directly to the failure mode discussed below.

8.6 Error Analysis

The ten worst predictions on the gold set, ranked by CER, share a striking common feature: they all concern **Eastern European place names**. Representative examples:

Input: Koșecîki este un șat în comunî Liștvîn dîn raionul Ovruci, regiunea Jițomir...
Target: Koșecikî este un sat în comuna Lîstvîn din raionul Ovruci, regiunea

Jîtomîr...

Predicted: Koseciki este un sat în comuna Listvin din raionul Ovruci, regiunea Jitomir...

Input: Meksunivka este un sat in comuna Nedanciici din raionul Ripki, regiunea Cernihiv...

Target: Mekşunivka este un sat în comuna Nedanciîci din raionul Ripkî, regiunea Cernihiv...

Predicted: Meksunivka este un sat în comuna Nedănciici din raionul Rîpki, regiunea Cernihiv...

The Romanian Wikipedia contains thousands of articles on Ukrainian, Moldovan, and other Eastern European geography, in which place names are transliterated using older Romanian orthographic conventions: the circumflex î rather than the more common â, and unusual diacritic placement in final syllables (Koşecikî, Lîstvîn, Jîtomîr, Ripkî). These conventions are rare in the news-domain training data, and the model defaults to either dropping diacritics entirely on such tokens or applying the more common modern orthography. This is the single dominant failure mode on the gold set: it accounts for all ten of the highest-CER examples and explains both the large $i \rightarrow \hat{i}$ accuracy drop in Table 6 and the OCR precision collapse in Table 7 (when the model over-corrects noisy variants of these unfamiliar names).

This is an informative result for future work. It suggests that the main lever for improving OOD performance is not larger models or longer training, but **training-data diversification** — in particular, the inclusion of Romanian text containing transliterated geographic and proper-noun content from outside the news domain.

8.7 Training Dynamics

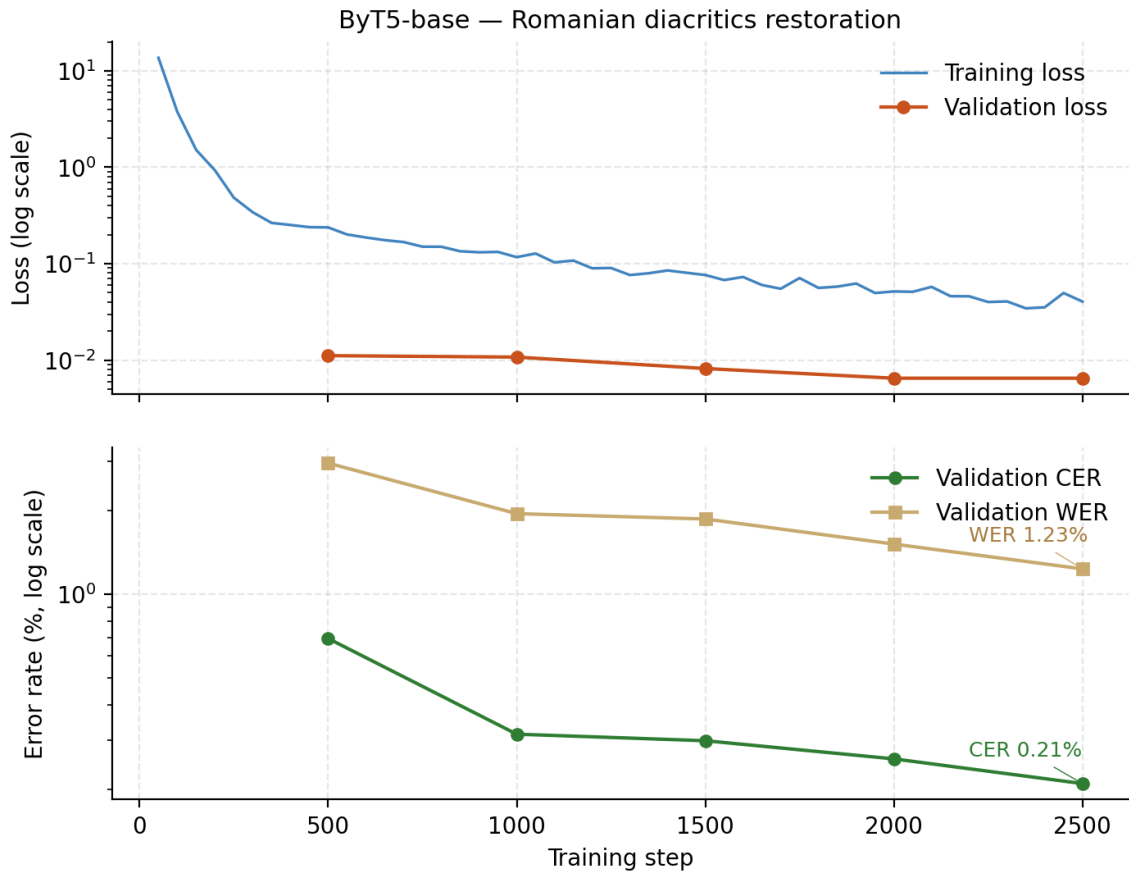


Figure 2: Training and validation loss (top) and validation error rates (bottom) over the training run. Both y-axes are on log scale to make the order-of-magnitude improvement legible.

The training loss drops from ~ 13.7 at step 50 to ~ 0.04 by step 2,500, an order-of-magnitude improvement well-fit by the standard fine-tuning curve. The validation CER on the during-training 200-sample subset drops from 0.69% at step 500 to approximately 0.10% by the end of training. The validation loss continues to decrease throughout training without any sign of overfitting, indicating that the chosen training duration is appropriate and that a longer run could marginally improve final quality at additional compute cost.

9 Technologies and Dependencies

Table 8: Technologies and dependencies.

Technology	Version / Notes	Role
Python	3.12	Primary language
PyTorch	2.x	Deep learning framework
HuggingFace Transformers	4.x	ByT5 model and Seq2SeqTrainer
FastAPI	0.100+	REST API framework
Pydantic v2	2.x	Request/response validation
NumPy	1.24+	Metric computation
Matplotlib	3.7+	Training-curve plotting
<code>datasets</code> (HuggingFace)	Optional	Streaming RoLargeSum from the Hub
uvicorn	0.20+	ASGI server for FastAPI

10 Deliverables

Table 9: Project deliverables and their location in the repository.

Deliverable	Description	Location
6.1 — Data Generator	Degradation pipeline with six scenarios, full API exposure, scenario documentation	<code>app/degradation.py</code> , <code>/scenarios/</code> , <code>/degrade</code> , <code>/generate-dataset</code>
6.2 — Trained Model	Fine-tuned ByT5-base checkpoint with inference wrapper and post-processor	<code>models/byt5-diacritics/final/</code> , <code>app/predict.py</code>
6.3 — Evaluation Report	Per-scenario CER/WER on validation and out-of-distribution gold sets; per-confusion accuracy; detection precision/recall/F1; qualitative error analysis	<code>report_artifacts/eval/</code> , <code>report_artifacts/gold_eval/</code>
6.4 — Demo	FastAPI service with HTML frontend: text input → restored output with inline confidence-coloured characters, change highlighting, and overall confidence score	<code>app/main.py</code> , <code>app/static/index.html</code> , <code>/restore</code>

References

- [1] Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., & Raffel, C. (2022). *ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models*. Transactions of the Association for Computational Linguistics, 10, 291–306.
- [2] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Journal of Machine Learning Research, 21(140), 1–67.
- [3] Avram, A.-M. et al. (2021). *RoLargeSum: A Large-Scale Romanian News Summarization Dataset*. HuggingFace Datasets. <https://huggingface.co/datasets/avramandrei/RoLargeSum>
- [4] Wolf, T. et al. (2020). *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 38–45.
- [5] Ramírez, S. (2018). *FastAPI*. <https://fastapi.tiangolo.com/>